
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON FANG - Requirements & Grading Criteria

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON FANG

REQUIREMENTS & GRADING CRITERIA

TARGET: NorthPharma smart parking barrier (smart-city lane controller)

ARTIFACT: ACT-IX.bin / ACT-IX.uf2 (compromised)

CREW: FROSTLINE

OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma runs the city's controlled edge, and its smart parking barrier built on a Pico 2 is the node on that edge. A contractor called **FROSTLINE** planted a weapon in the node image: an inverted boom slam that forces the boom down on a magic command, an inverted safety mask that makes the interlock falsely report clear, a reserved-sector weapon marker that re-arms the weapon on every boot, and an inverted barrier command authorization verdict. Operative **NIGHTINGALE** recovered the compromised image as `ACT-IX.bin`.

Students are the reverse-engineering reserve. They reverse engineer `ACT-IX.bin` with Ghidra, find and patch all four defects, defeat the CoreDebug `DHCSR` anti-debug under GDB to observe the marker write, export a corrected image, flash it to a real Pico 2, and prove the corrected behavior on the breadboard. The machine check is `scripts/verify_ctf.py`.

The challenge is a standalone capstone exercise and contains no answer, constant, address, bug, or patch belonging to any other course assignment.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the monitor loop.
- Locate four corrupted bytes: a boom slam gate, a safety-mask gate, a reserved-sector marker gate, and an authorization verdict branch.
- Analyze `cbz` and `cbnz` condition semantics and branch inversion.
- Explain why a local weapon that overrides the output makes an authenticated command path irrelevant, and why physical safety does not depend on the cipher.
- Explain why a controller that masks its own safety interlock turns a lane into a hazard behind the appearance of routine state.
- Explain why reserved-flash state survives a firmware reflash.
- Read CoreDebug `DHCSR`, explain the anti-debug trap, and defeat it under GDB.
- Explain why authentication is not authorization and why a verdict must be verified before the command is applied.

Students must use only the course concepts: ARM registers, stack behavior, USB-CDC and UART consoles, GDB, Ghidra static analysis and binary patching, vector tables, reset startup, XIP, Thumb addressing, condition-code analysis, stateful security, and the Argon2id plus XChaCha20-Poly1305 authenticated envelope.

Deliverables Checklist

#	Deliverable	Format	Criterion
1	Ghidra project screenshot	PNG/JPG	Task 1
2	Vector table and boot table	Inside <code>ACT-IX-Answers.md</code>	Task 1
3	<code>main</code> and monitor-loop table	Inside <code>ACT-IX-Answers.md</code>	Task 1
4	Module map	Inside <code>ACT-IX-Answers.md</code>	Task 1
5	Boom slam evidence and patch	Inside <code>ACT-IX-Answers.md</code>	Task 2
6	Safety mask evidence and patch	Inside <code>ACT-IX-Answers.md</code>	Task 3
7	Anti-debug GDB proof, reserved-sector evidence, and patch	Inside <code>ACT-IX-Answers.md</code>	Task 4
8	Barrier command authorization evidence and patch	Inside <code>ACT-IX-Answers.md</code>	Task 5
9	<code>ACT-IX_fixed.bin</code>	BIN file	Task 6
10	<code>ACT-IX_fixed.uf2</code>	UF2 file	Task 6
11	Hardware proof and reflection	Inside <code>ACT-IX-Answers.md</code>	Task 6

Required Tools and Equipment

Tool	Purpose
Raspberry Pi Pico 2	Isolated target node
Debug Probe (OpenOCD)	SWD connection for GDB inspection and the anti-debug work
arm-none-eabi-gdb	Runtime breakpoints, DHCSR clearing, and reserved-sector observation
Ghidra	Static analysis and binary patching
Python 3 with <code>uf2conv.py</code>	UF2 conversion and artifact checks
DHT11, 1602 I2C LCD, RYL998, IR receiver, SG90 servo, 3 LEDs, manual raise button	Breadboard hardware proof
<code>ACT-IX.bin</code> and <code>ACT-IX.uf2</code>	Supplied compromised artifacts

Console settings: **USB-CDC virtual COM port, 115200 baud, 8 data bits, no parity, 1 stop bit.** Radio UART settings: **UART1, 115200, network ID 18.**

Artifact Identity

The instructor-issued artifact hashes are:

```

ACT-IX.bin          dbdf7ff59c22dce3f008cbcdccf35eb97b97ff50be65a574befd39f05f0c6a32
ACT-IX.uf2          174a68a7e82af9d7eb5097bfb000743bfc8ac40cc05a91a477f9eb8e22b6d460
ACT-IX_fixed.bin    4b9438fd9157c859508523baec7e18bda3119b7c562d957872470268bf39f0d8
ACT-IX_fixed.uf2    d70c85965bd9fb13848a3302128b4c8e3a6801ed813786b4831a901039d0832e

```

The verifier checks the `ACT-IX.bin` and `ACT-IX_fixed.bin` hashes specifically, asserts the four fixed bytes, and requires that only those four offsets differ between the two `.bin` images. Both `.bin` images are 50,972 bytes and both `.uf2` images are 102,912 bytes.

Grading Rubric - Detailed Breakdown

Task 1: Setup and Initial Analysis (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronFang_Investigation</code> , raw binary import	One item off	Not set up
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor	Wrong language	Missing
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>	Wrong base	Missing
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>	One missing	Not found
[DOCUMENT] main and the barrier monitor state machine (<code>monitor_step</code>) addresses identified	1	<code>main 0x10000234</code> , <code>monitor_step 0x10006688</code>	One correct	Neither
[DOCUMENT] Module map identifies the boom, control, barrier_auth, implant, and monitor anchors	1	At least one correct anchor per module	Partial	Missing

Task 2: Bug #1 The Boom Slam (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the boom slam gate	5	Address and function (<code>implant_weapon_armed</code>) identified	Approximate	Not found
[DOCUMENT] Documented the boom slam that forces the boom down on the magic command	5	Weapon gate <code>0x20013CF6</code> , forced lowered boom, magic <code>IRON-FANG-SLAM-2026</code>	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the boom slam is not armed	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Explained why the slam forces the boom down regardless of the authorized state	3	Local override beside the authenticated command path, physical safety as a policy control	Vague	Missing

Task 3: Bug #2 The Safety Mask (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the safety mask gate	5	Address and function (<code>implant_safety_masked</code>) identified	Approximate	Not found
[DOCUMENT] Documented the masked safety loop that falsely reports clear	5	Mask gate <code>0x20013CF4</code> , <code>monitor_safety_clear</code> returns true while masked	Partial	Wrong
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so the safety loop is never masked	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained why a masked safety loop is a physical-safety failure	3	The interlock is part of the attack surface and the truth is a control	Vague	Missing

Task 4: Bug #3 The Weapon Marker (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the weapon marker gate	5	Address and inlined <code>implant_init</code> path identified	Approximate	Not found
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method	Partial	Wrong
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so no weapon marker is programmed to <code>0x103FF000</code>	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained the reserved sector <code>0x103FF000</code> and the weapon marker byte <code>0x57</code>	3	Marker, reserved sector, write-once first run	Vague	Missing

Task 5: Bug #4 The Barrier Command Authorization (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the barrier command authorization branch	5	Address and function (<code>control_handle_frame</code>) identified	Approximate	Not found

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so failed and replayed authorizations are rejected	7	Byte 0xB9 changed to 0xB1	Wrong byte	Not patched
[DOCUMENT] Explained why an unauthenticated or replayed barrier command must be rejected	3	The applied command must see only an authorized verdict	Vague	Missing

Task 6: Export and Verify (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[PATCH] Exported ACT-IX_fixed.bin from Ghidra	2	Valid patched binary	Corrupt	Not submitted
[PATCH] Converted to ACT-IX_fixed.uf2 with the correct base and family	2	--base 0x10000000 --family 0xe48bff59	Wrong flags	Not submitted
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown	Partial proof	No proof
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four	Partial	Missing

Common Pitfalls

Pitfall	Consequence	Avoidance
Reading the boom slam gate backwards	The boom is still slammed and the boom is forced down	Neutralize only on the clear-gate branch (cbz , 0xB1)
Reading the safety mask gate backwards	The interlock is still masked	Neutralize only when the gate is clear (cbz , 0xB1)
Confusing cbz and cbnz	The boom still slams, the loop still lies, or the marker is still written	Neutralize only when the gate is clear (cbz , 0xB1)
Patching the low byte of a gate	The condition code never changes	Patch the high byte of the branch
Searching for a standalone implant_infect symbol	Cannot find the inlined gate	Look inside implant_init
Confusing the weapon gate with the marker gate	Both sit in the implant functions at different addresses	Patch the weapon gate first, then the marker gate
Patching the shipped image before observing the write	You never prove the weapon marker write	Defeat DHCSR under GDB first, then patch the artifact

Pitfall	Consequence	Avoidance
Fabricating the GDB session	Verification fails	Show the command sequence and the real observed code path
Treating the anti-debug as a defect to patch	Wasted effort; it is identical in both images	Defeat it in a scratch copy or with GDB, then patch the real defect
Missing that the authorization branch is a verdict	Unauthenticated commands still reach the applied command and zone	Accept only when the verdict is true (<code>cbz</code> to reject, <code>0xB1</code>)
Forgetting that the fix is also a policy	The device still fails closed on a lost link	Make the barrier fail safe to the raised posture when the link is lost
Forgetting UF2 conversion	Raw binary will not flash	Use <code>uf2conv.py</code> with family <code>0xe48bff59</code>

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 cabinet temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Boom barrier servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	DENIED, 220 to 330 ohm to GND
Yellow LED	anode	GP17	PASS PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	OPEN, 220 to 330 ohm to GND
Manual raise button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Memory Map Reference

Region	Address	Purpose
Bootrom	0x00000000	Immutable boot code
Flash/XIP	0x10000000	Vector table, code, rodata, data image
SRAM	0x20000000	Stack and writable state
CoreDebug_DHCSR	0xE00EDF0	Anti-debug register read by the implant
Implant reserved sector	0x103FF000	Weapon marker target (last flash sector)
Implant tick counter	0x20013710	Incremented once per <code>implant_tick</code>
Implant weapon count	0x20013714	Number of weaponize operations this boot
Implant armed flag	0x20013CF2	Set when the implant arms
Implant weaponized flag	0x20013CF7	True while the boom slam is active
Implant safety-masked flag	0x20013CF5	True while the safety loop is masked
Implant weapon gate	0x20013CF6	Gates the boom slam
Implant mask gate	0x20013CF4	Gates the safety-loop mask
Implant marker gate	0x20013CF3	Gates the reserved-sector weapon marker write
Control ready gate	0x20013CF0	Gates the sealed barrier command path
Applied command	0x20013CEF	Command after a true verdict
Applied zone	0x20013CE2	Zone after a true verdict
Authorization ready gate	0x20013CEC	Gates the authorization check
Auth state record	0x200136CC	Anti-replay and state-tag record
Control field key	0x200136E8	Derived field key for the envelope

The VA of any file offset is the file offset plus `0x10000000`.

Deadline & Submission

- Create a folder containing the Ghidra screenshot, `ACT-IX_fixed.bin`, and `ACT-IX_fixed.uf2`.
- Write all written answers in `ACT-IX-Answers.md` inside that folder.
- Include the output of `python scripts/verify_ctf.py`.
- ZIP the folder as `lastname-firstname-ACT-IX.zip`.
- Submit the ZIP before the posted deadline; late submissions lose 10 percent per day.

Grade Scale

Grade	Percentage	Points
A+	97-100%	97-100
A	93-96%	93-96
A-	90-92%	90-92
B+	87-89%	87-89
B	84-86%	84-86
B-	80-83%	80-83
C	70-79%	70-79
F	0-69%	0-69%

Academic Integrity

Use only the supplied Pico 2 and firmware. Do not connect the exercise to an operational smart-city network, a parking control system, a building-management system, a public network, a military system, or a third-party device. This is a controlled, isolated educational exercise. All analysis and patches must be your own work; sharing binaries, addresses, keys, passphrases, or answers is a violation of the academic integrity policy.

Reference Material

Topic	Reference
ARM Cortex-M33 registers and stack	Course block 1
USB-CDC and UART console capture	Course block 2
Vector tables, reset startup, and XIP	Course block 3
Ghidra static analysis and binary patching	Course block 4
Physical weaponization and actuator abuse	Course block 5
Safety integrity and interlock masking	Course block 6
Reserved-flash persistence and boot re-install	Course block 7
CoreDebug DHCSR and anti-debug	Course block 8
Argon2id and XChaCha20-Poly1305 authenticated envelope	Course block 9